Les tableaux en Python (Listes)

- Listes 1D
- Chaînes de caractères
- Listes 2D

Coordinatrice : Cherifa BEN KHELIL
cherifa.ben-khelil@efrei.fr
Support du cours: Asma Gabis

P1, P1-BN, P1-P, P1-BDX – 2024/2025

01

Les listes 1D

01

Les listes 1D

Définition

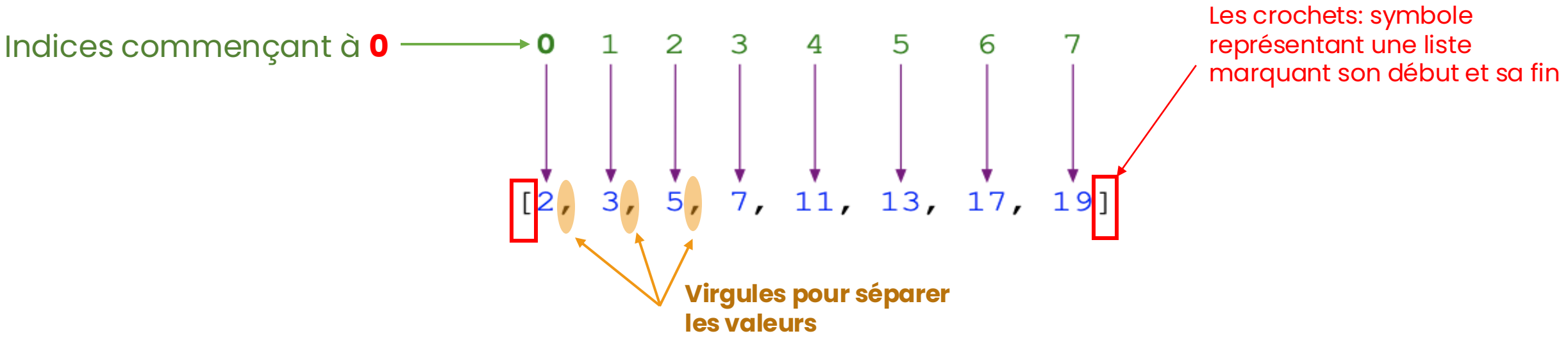
- Un tableau est représenté en Python par une liste.
- Une liste : Une variable structurée regroupant des données **les unes à la suite des autres**.
- Chaque valeur est identifiée par **sa position** dans la liste.
- **Exemples** :

```
moyennes = [15.3, 18.9, 12.5, 9.6, 17.1] # Liste de valeurs de type float représentant des moyennes

L = [ 12, -2, 34, 11, 897] # Liste de valeurs de type int

prenoms = ["Jean", "Marie", "Amine", "Pedro"] #Liste de valeurs de type str représentant des prénoms
```

Les listes 1D

Représentation d'une liste

- Python présente les listes comme une suite de valeurs séparées par des virgules, entourées par des crochets. Un indice (position) est associé à chaque valeur de la liste.
- Les indices sont attribués dans **un ordre séquentiel croissant** de gauche à droite et commencent par **ZERO (0)**



Les listes 1D

Création d'une liste

Création d'une liste vide :

Il est possible de créer une liste vide et lui ajouter des valeurs plus tard

```
L = []
```

Nom de la variable liste

Crochets vides

Création d'une liste avec des valeurs :

Il est également possible de créer une liste pré-remplie

Crochets symboles d'une liste

```
nombres_impairs = [1, 3, 5, 7, 9]
```

Nom de la variable liste

Valeurs de la liste séparées par des virgules

Les listes 1D

Accès aux éléments d'une liste

On accède à un élément de la liste en utilisant le nom de la liste et la position de cet élément entre crochets.

```
nom_liste[indice_element]
```

Exemple :

```
premiers = [2, 3, 5, 7, 11, 13, 17, 19]
print("La valeur à l'indice 3 est :", premiers[3]) # À noter que c'est la 4ème valeur de la liste
print("La 5ème valeur de cette liste est :", premiers[4]) # Car la numérotation commence à partir de 0
```

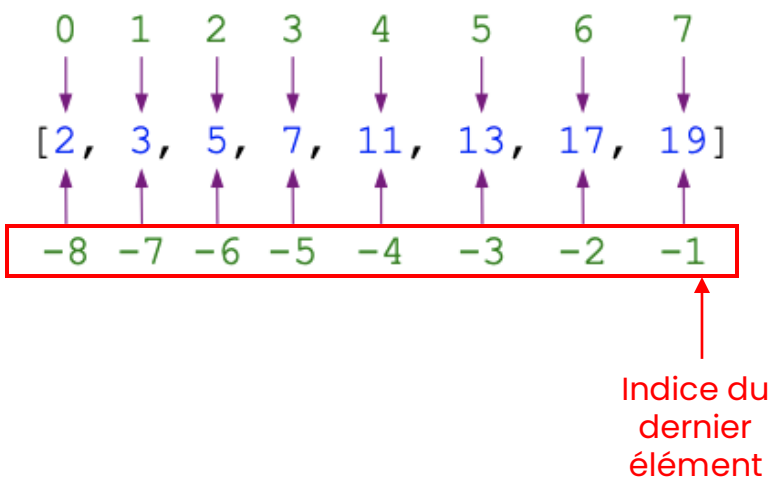
```
La valeur à l'indice 3 est : 7
La 5ème valeur de cette liste est : 11
```

Les listes 1D

Les indices négatifs

En Python, les valeurs d'une liste sont également accessibles de la fin (droite) vers le début (gauche) en utilisant des indices négatifs. Ainsi:

- -1 est l'indice du dernier élément
- - 2 est l'indice de l'avant dernier élément
- ….



Exemple :

```
fruits = ["banane", "poire", "framboise", "pomme"]
print("Le dernier élément de cette liste est :", fruits[-1])
print("Le 3ème élément en partant de la fin de la liste est :", fruits[-3])
```

```
Le dernier élément de cette liste est : pomme
Le 3ème élément en partant de la fin de la liste est : poire
```

Les listes 1D

La taille d'une liste

- En Python, il est possible d'avoir à tout moment le nombre de valeurs d'une liste
- Le nombre de valeurs représente la taille logique de la liste
- La fonction qui donne la taille est : **len(nom_liste)**

Exemple :

```
fruits = ["banane", "poire", "framboise", "pomme"]
print("la taille de la liste fruits est :", len(fruits))
moyennes = [15.3, 18.9, 12.5, 9.6, 17.1]
print("la taille de la liste moyennes est :", len(moyennes))
```

```
la taille de la liste fruits est : 4
la taille de la liste moyennes est : 5
```

Les listes 1D

Parcourir une liste

Parcourir une liste via les indices

- Grâce à une boucle **for**
- Utilisation de l'opérateur « **in** » sur les valeurs des indices des éléments d'une liste.

indice : prend les valeurs de
0 à **len(nom_liste) - 1**

```
for indice in range (len(nom_liste)) :
```

Exemples :

```
fruits = ["banane", "poire", "framboise", "pomme"]
for indice in range (len(fruits)):
    print(fruits[indice], end=" ")
```

```
banane poire framboise pomme
```

Les listes 1D

Parcourir une liste

Parcourir une liste via les indices

ATTENTION !!! Lors du parcours d'une liste, il ne faut **JAMAIS** dépasser sa taille.

Les indices d'une liste *nom_liste* de taille *len(nom_liste)* sont entre **0** et **len(nom_liste) – 1**

Toute tentative d'accès à une case se trouvant en dehors de cet intervalle engendre une **erreur**

Exemples :

```
fruits = ["banane", "poire", "framboise", "pomme"]
print(fruits[len(fruits)])
```

La case d'indice len(fruits) n'existe pas

```
Traceback (most recent call last):
  File "/……", line 20, in <module>
    print(fruits[len(fruits)])
IndexError: list index out of range
```

Les listes 1D

Parcourir une liste

Parcourir une liste via un itérateur

- Grâce à une boucle **for**
- En utilisant l'opérateur d'appartenance « **in** » à une liste

for valeur in nom_liste:

Représente une valeur (contenu de la case) de la liste

Exemple :

```
fruits = ["banane", "poire", "framboise", "pomme"]
for valeur in fruits:
    print(valeur, end=" ")
```

```
banane poire framboise pomme
```

Les listes 1D

Affichage des éléments d'une liste

En Python, il est possible d'afficher une liste en bloc en utilisant la fonction **print()** comme pour une simple variable.

```
print(nom_liste)
```

Mettre directement le nom de la liste sans indices

Exemple :

```
fruits = ["banane", "poire", "framboise", "pomme"]
print(fruits)
```

```
['banane', 'poire', 'framboise', 'pomme']
```

Style d'affichage unique: sous forme d'une liste
- Crochets en début et fin de la liste
- Virgules pour séparer les valeurs
- Ici quotes car valeurs de type str

Les listes 1D

Ajout d'un élément à la liste

- La notion de taille physique n'existe pas sur les listes
- Il est possible d'ajouter à tout moment des valeurs dans la liste
- La taille de la liste est automatiquement mise à jour suite à un ajout d'une valeur
- La méthode **append()** permet d'ajouter une valeur mais **toujours à la fin** de la liste

nom_liste.append(valeur_a_ajouter)

Exemple :

```
fruits = ["banane", "poire", "framboise", "pomme"]
print("la taille de la liste fruits est :", len(fruits))
fruits.append("melon")
print("la nouvelle taille de la liste fruits est :", len(fruits)) # la taille de la liste est mise à jour
```

```
la taille de la liste fruits est : 4
la nouvelle taille de la liste fruits est : 5
```

Les listes 1D

Suppression d'un élément de la liste

- Il s'agit d'une suppression d'un élément par son indice
- La fonction permettant la suppression est **del** `del nom_liste[indice_valeur_a_supprimer]`
- Après la suppression les éléments sont automatiquement décalés vers la gauche
- La taille de la liste est mise à jour automatiquement

Exemple :

```
fruits = ["banane", "poire", "framboise", "pomme"]
print("la taille de la liste fruits est :", len(fruits))
fruits.append("melon")
print("la taille de la liste fruits après l'ajout d'un fruit est :", len(fruits))
del fruits[2]
print("la taille de la liste fruits après la suppression du fruit à l'indice 2 :", len(fruits))
```

```
la taille de la liste fruits est : 4
la taille de la liste fruits après l'ajout d'un fruit est : 5
la taille de la liste fruits après la suppression du fruit à l'indice 2 : 4
```

Les listes 1D

Modification d'un élément de la liste

Pour modifier une valeur dans une liste se trouvant à une position donnée, il suffit d'affecter la nouvelle valeur à la case de la même position.

```
nom_liste[indice_valeur_a_modifier] = nouvelle_valeur
```

Exemple :

```
fruits = ["banane", "poire", "framboise", "pomme"]
print(fruits)

fruits[2] = "fraise"
print(fruits)

fruits[-1] = "abricot"
print(fruits)
```

```
['banane', 'poire', 'framboise', 'pomme']
['banane', 'poire', 'fraise', 'pomme']
['banane', 'poire', 'fraise', 'abricot']
```

Les listes 1D

Les slices en Python

- Permettent le découpage d'une liste afin d'en extraire une partie.
- Slices possibles :
  - nom_liste [**n** : **p**] retourne la liste des éléments numérotés de **n** à **p – 1**.
  - nom_liste [: **p**] retourne la liste des éléments numérotés de **0** à **p – 1** (tous les éléments jusqu'au (p-1) nième.)
  - nom_liste [**p** :] retourne la liste de tous les éléments allant de l'indice **p** à **len(nom_liste)-1**.

Les listes 1D

Les slices en Python

Exemple :

```
fruits = ["banane", "poire", "framboise", "pomme"]

print(fruits[1:3]) # valeurs des positions 1 et 2

print(fruits[:3]) # valeurs des positions 0, 1 et 2

print(fruits[2:]) # valeurs des positions 2 et 3

print(fruits[:]) # toutes les valeurs de la liste
```

```
['poire', 'framboise']
['banane', 'poire', 'framboise']
['framboise', 'pomme']
['banane', 'poire', 'framboise', 'pomme']
```

Les listes 1D

Autres opérations sur les listes

Concaténation :

Liste1 + **Liste2** = Longue liste où **Liste2** est insérée à la **fin** de **Liste1**

Opérateur de concaténation

Exemple :

```
liste1 = [1, 2, 3, 4]
liste2 = [5, 6, 7]
liste = liste1 + liste2
print(liste)
```

```
[1, 2, 3, 4, 5, 6, 7]
```

Les listes 1D

Autres opérations sur les listes

Répétition :

Nombre * **Liste** =

Opérateur de répétition

Liste plus longue avec comme contenu les valeurs de **Liste** répétées **Nombre** fois

Exemple :

```
liste1 = [1, 2]
liste = 3 * liste1
print(liste)

# Pour initialiser une liste, on peut utiliser ceci
L = [0] * 10
print(L)
```

```
[1, 2, 1, 2, 1, 2]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Les listes 1D

Autres opérations sur les listes

Tester l'existence d'une valeur dans une liste :

Il est possible de chercher une valeur dans une liste en utilisant l'opérateur d'appartenance « **in** »

```
if valeur_recherchee in nom_liste :
```

Exemple :

```
prenoms = ["Jean", "Marie", "Amine", "Pedro"]

print("Donner votre prénom :")
prenom = input()
if prenom in prenoms :
    print("Votre prénom existe déjà dans la liste")
else:
    print("Votre prénom n'existe pas dans la liste")
```

```
Donner votre prénom :
Pedro
Votre prénom existe déjà dans la liste
```

```
Donner votre prénom :
Julie
Votre prénom n'existe pas dans la liste
```

02

Les chaînes de caractères

02

Les chaînes de caractères

Manipulation des chaînes de caractères en Python

- Les chaînes de caractères peuvent être considérées comme des listes (de caractères) particulières.
- Chaque caractère est accessible par sa position dans la chaîne de caractères
- Les indices des caractères commencent à **0**
- Ainsi :
  - Il est possible de parcourir une chaîne de caractères comme une liste
  - Il est possible d'utiliser les slices sur les chaînes de caractères
  - Il est possible d'accéder à un caractère d'une chaîne de caractère via son indice
- **MAIS** :
  - Il n'est **PAS** possible de modifier un caractère d'une chaîne de caractère

Les chaînes de caractères

Manipulation des chaînes de caractères en Python

Exemples :

```
mon_prenom = "Nadia"
print("Mon prénom est :", mon_prenom)
```

```
Mon prénom est : Nadia
```

```
mon_prenom = "Nadia"

# Convertir (caster) une str en list
list_mon_prenom = list(mon_prenom)
print(list_mon_prenom)
```

```
['N', 'a', 'd', 'i', 'a']
```

```
mon_prenom = "Nadia"

# Convertir (caster) une str en list
list_mon_prenom = list(mon_prenom)
# Afficher les éléments de la liste
for lettre in list_mon_prenom:
    print(lettre)
```

```
N
a
d
i
a
```

```
mon_prenom = "Nadia"
mon_prenom[1] = 'b'
print(mon_prenom)
```

```
mon_prenom[1] = 'b'
TypeError: 'str' object does not support item
assignment
```

Les chaînes de caractères

Codage ASCII

- Codage « universel » permettant d'associer à chaque caractère un numéro
- Les correspondances caractère – numéro de tous les caractères sont réunies dans la table ASCII

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Source : wikipedia

Les chaînes de caractères

Codage ASCII

- Ce qu'il faut retenir de la table ASCII:
  - À chaque caractère est associé un code hexadécimal et son équivalent en décimal
  - Les lettres majuscules ont des codes ASCII différents des lettres minuscules
  - Les codes des lettres majuscules sont dans l'intervalle décimal [65, 90]
  - Les codes des lettres minuscules sont dans l'intervalle décimal [97, 122]
  - La différence entre une lettre majuscule d'une lettre minuscule est toujours 32

Les chaînes de caractères

Codage ASCII en Python

Code ASCII d'un caractère

- En Python, la fonction **ord(caractère)** donne le code décimal d'un caractère.

Exemple :

```
print("le code de la lettre a est :", ord("a"))
print("le code de la lettre A est :", ord("A"))
print("le code du caractère # est :", ord("#"))
```

```
le code de la lettre a est : 97
le code de la lettre A est : 65
le code du caractère # est : 35
```

Les chaînes de caractères

Codage ASCII en Python

Caractère correspondant à un code ASCII

- En Python, la fonction **chr(code)** donne le caractère correspondant au code.

Exemple :

```
print("Le caractère correspondant au code 65 est :",chr(65))
print("Le caractère correspondant au code 97 est :",chr(97))
print("Le caractère correspondant au code 35 est :",chr(35))
```

```
Le caractère correspondant au code 65 est : A
Le caractère correspondant au code 97 est : a
Le caractère correspondant au code 35 est : #
```

Les chaînes de caractères

Codage ASCII en Python

Cas d'utilisation

Trouver le successeur d'un caractère donné

```
car="e"
code_car=ord(car)
successeur_car=chr(code_car+1)
print("Le successeur de",car,"est :",successeur_car)
```

```
Le successeur de e est : f
```

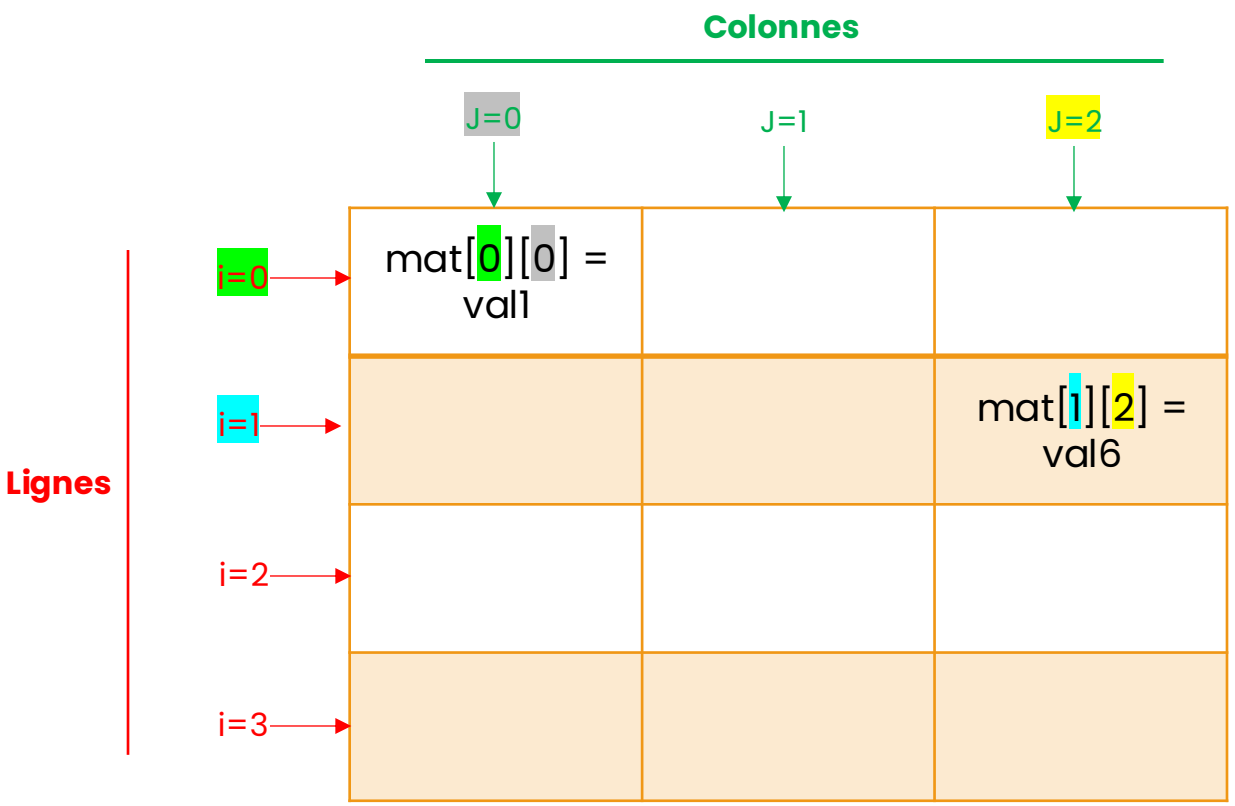
03

Les listes 2D

03

Les listes 2D

Représentation générale : Rappel



Les listes 2D

Représentation en Python

- Une liste à 2D (matrice) = Une liste 1D où chacune de ses valeurs est elle-même une liste 1D séparées par des virgules.

  Liste2D = [[sous_liste1] , [sous_liste2] , … , [sous_listen]]

- Ainsi:
  - L'utilisation d'un seul indice sur la liste principale Liste2D affiche une ligne complète de la liste 2D.

    Exemple :

```
liste2D = [[1, 2, 3],
           [4, 5, 6]
          ]
print(liste2D[0])
```

```
[1, 2, 3]
```

Les listes 2D

Représentation en Python

- Pour accéder à une valeur stockée dans une liste 2D, il faut:
  - D'abord accéder à la ligne par le premier indice
  - Puis en utilisant un deuxième indice, accéder à une colonne

Exemple :

```
liste2D = [[1, 2, 3],
           [4, 5, 6]
          ]
print(liste2D[0][1])
print(liste2D[1][1])
```

```
2

5
```

Les listes 2D

Création d'une liste 2D

- Quelques créations de listes 2D pré-remplies
- La fonction print() permet d'afficher les valeurs d'une liste2D

```
# Création et initialisation d'une liste 2D
# nombre de ligne = nombre de colonnes = 3
liste2D = [[1, 2, 3],
           [4, 5, 6],
           [7, 8, 9]
           ]
```

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Les listes 2D

Parcourir une liste 2D

Via les indices

- Deux boucles imbriquées
  - Une boucle externe qui va parcourir les lignes (taille de la liste 2D)
  - Une boucle interne qui va parcourir les colonnes (taille de chaque valeur de la liste 2D)

```
# Initialisation d'une liste 2D
liste2D = [[1, 2, 3],
           [4, 5, 6]
           ]
# Parcourir les valeurs de la liste (valeurs = nombre de lignes)
for i in range(len(liste2D)):
    # La taille de chaque ligne représente le nombre de colonnes
    for j in range(len(liste2D[i])):
        # Affichage des valeurs de la ligne i séparées par des espaces
        print(liste2D[i][j], end=' ')
    # retour à la ligne avant l'affichage de la prochaine ligne
    print()
```

```
1 2 3
4 5 6
```

Les listes 2D

Parcourir une liste 2D

Via les itérateurs

- Deux boucles imbriquées
  - Une boucle externe qui va parcourir les lignes (chacune des valeurs de la liste 2D)
  - Une boucle interne qui va parcourir les colonnes (Les valeurs de la liste sur une case)

```
# Initialisation d'une liste 2D
liste2D = [[1, 2, 3],
           [4, 5, 6]
           ]
# Parcourir chaque case (une case contient une liste)
for row in liste2D:
    for elem in row:
        print(elem, end=' ')
    print()
```

1	2	3
4	5	6

Les listes 2D

Remplir une liste 2D

Exemple de remplissage par des données saisies par l'utilisateur

```
# Initialisation d'une liste 2D
liste2D = []
nb_lig = int(input("Saisir un nombre de lignes : "))
nb_col = int(input("Saisir un nombre de colonnes : "))
for i in range(nb_lig):
    # déclarer une liste 1D vide qui sera remplie par l'utilisateur
    L=[]
    for j in range(nb_col):
        val = int(input("Saisir une valeur : "))
        # Ajouter la valeur saisie à la liste L
        L.append(val)
    # une fois nb_col valeurs sont saisies, elles sont ajoutées à liste2D
    # Boucler pour saisir la prochaine ligne
    liste2D.append(L)

print(liste2D)
```

```
Saisir un nombre de lignes : 3
Saisir un nombre de colonnes : 2
Saisir une valeur : 1
Saisir une valeur : 2
Saisir une valeur : 3
Saisir une valeur : 4
Saisir une valeur : 5
Saisir une valeur : 6
[[1, 2], [3, 4], [5, 6]]
```

04

Activité

04

Activité

1. Écrire un programme Python qui :
   - demande à l'utilisateur le nombre de valeurs N qu'il souhaite saisir
   - permet de remplir une liste 1D par N valeurs saisies au clavier

2. Modifier le programme précédent afin que l'utilisateur puisse saisir des valeurs jusqu'à ce qu'il décide lui-même de s'arrêter. Afficher le nombre de valeurs saisies.

3. Afficher les valeurs d'une liste 1D séparées par des points-virgules (Attention au dernier élément)